

Vision-Guided Robotic Pick-and-Place

Computer Vision, 3D Object Localization and Robotic Manipulation in Simulation

Rabah Bouguezel

August 2026

<https://github.com/midnight004/Vision-Guided-Robotic-Pick-and-Place>

1. Project Overview

This project is an autonomous factory sorting cell simulated in MuJoCo. A conveyor belt feeds a stream of random products into the cell one at a time. An overhead RGB-D camera perceives each item, the system localizes it in 3D and classifies its color, and a Franka Emika Panda arm picks the item and places it into the matching color bin. Products whose color is not one of the four known categories are routed to a reject (trash) bin.

The cell exercises a complete robotics-and-vision pipeline end to end:

```
Conveyor feed -> RGB-D camera -> Object detection -> 3D localization
-> Color classification (known bin vs. reject) -> Inverse kinematics
-> Contact-triggered physics grasp -> Pick and place into bins
```

Each episode draws a randomized mix of known-color and unknown-color products, in random order and orientation, so no two runs are identical.

2. Technology Stack

Purpose	Technology
Physics / simulation	MuJoCo 3.11.0
Robot model	Franka Emika Panda (7-DOF), MuJoCo Menagerie
Language / runtime	Python 3.12 in a virtual environment
Computer vision	OpenCV, segmentation-mask rendering, HSV analysis
Numerics	NumPy, SciPy
Label textures	Pillow
Configuration	PyYAML
Optional deep-learning path	PyTorch, Ultralytics YOLOv8

The development GPU is AMD, so the deep-learning detection path is optional. The active perception path is the segmentation plus color pipeline, which is reliable and CPU-light.

3. Installation and Running

```
python3.12 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Run with the interactive 3D viewer:

```
python scripts/run_pipeline.py --episodes 10 --objects 6
```

Headless evaluation (metrics only):

```
python scripts/run_pipeline.py --headless --episodes 6 --objects 6
```

4. Repository Layout

```
assets/ scene.xml, franka/ (Panda model), textures/ (labels)
config/ per-module YAML configuration
```

```
scripts/ run_pipeline.py (entry point), make_labels.py, make_report.py
src/ simulation, camera, detection, localization, tracking,
robot_control, task_logic, evaluation, utils
```

Module	Responsibility
simulation/environment.py	Model loading, physics stepping, RGB-D + segmentation rendering, conveyor feed, grasp welds, object parking
detection/detector.py	Segmentation / color / YOLO backends and HSV color classifier
localization/localizer.py	Depth back-projection to world coordinates, pick-zone filter
robot_control/arm_controller.py	Damped least-squares IK, Cartesian moves, poses
robot_control/gripper_controller.py	Gripper open/close and contact-triggered weld grasp
robot_control/pick_place.py	Pick-and-place state machine and motion heights
scripts/run_pipeline.py	Episode queue, perception + motion orchestration, landing checks

5. The Simulation Scene

The scene is defined in assets/scene.xml. Global physics use a 0.002 s timestep, the implicitfast integrator, and a pyramidal contact cone with impratio 1 for stability with small round objects. Lighting is diffuse only, which keeps object colors accurate for the color classifier.

Five sorting bins. Corner bins are 0.08 half-size, the trash bin is 0.07, and all walls are 0.05 tall:

Bin	Position (x, y, z)
Red	(0.30, -0.30, 0.30)
Blue	(0.30, 0.30, 0.30)
Green	(0.62, -0.30, 0.30)
Yellow	(0.62, 0.30, 0.30)
Trash	(0.46, 0.31, 0.30)

Fourteen products are parked off-screen and fed one at a time:

- Known colors (8): red_box, red_can, blue_box, blue_capsule, green_cylinder, green_box, yellow_sphere, yellow_bottle
- Unknown colors (6), routed to trash: purple_box, orange_sphere, white_cylinder, black_box, purple_cylinder, orange_box

An equality block defines 14 weld constraints (hand to each object), inactive by default and activated at runtime only when the gripper closes on an object.

6. Perception: Detection and Color Classification

Detection supports three backends: segmentation (default, using MuJoCo segmentation rendering for pixel-accurate masks), pure color thresholding (fallback), and a custom YOLOv8 model (wired in but not the active path). Raw color thresholding is unreliable under the lighting, so segmentation provides the mask and color is judged from the RGB pixels inside it, separating 'where is the object' from 'what color is it'.

For each object mask, only saturated, well-lit pixels vote (saturation > 40, value between 25 and 253). If fewer than 8% of pixels are colored, the object is labeled unknown (captures white, black, grey). Otherwise pixels are binned by hue:

Hue range (OpenCV 0-179)	Category
<= 10 or >= 160	red
11 - 23	unknown (orange)
24 - 35	yellow
36 - 85	green
86 - 132	blue
>= 133	unknown (purple / magenta)

The winning category must hold at least 45% of the colored pixels, otherwise the object is unknown. That category, not the object's true identity, decides the destination bin, which is what lets the arm send a novel purple or orange item to trash. During perception the arm moves to a scan pose so it does not occlude the camera.

7. 3D Localization

For each detection, the masked region's depth is read and the pixel is back-projected through the pinhole model with the principal point at the image center and focal length from the camera field of view (640x480):

$$\begin{aligned}X &= (u - cx) * Z / fx \\Y &= (v - cy) * Z / fy \\Z &= \text{depth}\end{aligned}$$

The camera-frame point is transformed to world coordinates using the known camera extrinsics (exact in simulation). A pick-zone filter (roughly x in [0.30, 0.70], y in [-0.26, 0.12], z in [0.28, 0.50]) discards bin positions and stray geometry so only the staged product is targeted.

8. Robot Control

The arm uses damped least-squares inverse kinematics with a downward-orientation constraint for top-down grasps. A Cartesian move succeeds when the end-effector reaches within 0.08 m of the target. The home pose (qpos) is [0, -0.785, 0, -2.356, 0, 1.571, 0.785] and the scan pose is [0, -1.5, 0, -2.5, 0, 1.5, 0.785].

The pick-and-place state machine runs: APPROACH, DESCEND, GRASP, LIFT, TRANSPORT, LOWER, RELEASE, RETREAT. Motion heights:

Parameter	Value	Purpose
Approach height	0.15	Above the object before descending
Grasp Z offset	0.058	Hand-to-fingertip offset
Lift height	0.56	Absolute Z after lifting
Transport height	0.56	Object hangs ~9 cm below and clears the 0.40 m walls
Place height offset	0.10	Release above bin floor so item drops in cleanly
Retreat height	0.56	Absolute Z after release

The grasp step retries once if it closes on nothing, and a failed motion returns the arm to the scan pose so the next item starts clean, preventing cascading failures.

9. Grasping Mechanism

The final mechanism is contact-triggered weld grasping:

- The gripper closes physically on the object.
- The controller identifies the object between the fingers by horizontal (XY) distance within a vertical window beneath the hand; the object center sits about 8-9 cm below the end-effector reference, so a naive 3D check would miss it.
- A MuJoCo weld equality constraint (hand to object) is activated at the current relative pose, computed as $\text{hand}^{-1} * \text{object}$ and written into the constraint data.
- The weld holds the object rigidly through lift and transport, like a firm grip.
- Opening the gripper deactivates the weld and the object drops into the bin.

This is genuine constraint-based physics, not a teleport or attach hack, and it is the standard technique in MuJoCo manipulation research. It eliminates the slipping and lag that pure friction grasping produced for small objects during motion.

10. Conveyor Feed and Sorting

Each product spawns at the belt entrance near (0.5, -0.55, 0.35) with a random X offset and yaw, is driven forward at 0.45 m/s to the staging gate near (0.5, -0.12, 0.33), then its velocity is zeroed and it settles to a full stop so the perceived position is stable. Feeding one item at a time mimics a real sorting line.

Detected color	Bin
red	Red
blue	Blue
green	Green
yellow	Yellow

unknown	Trash
---------	-------

11. Evaluation

Evaluation is honest: success is not 'the arm finished moving' but whether the object physically ended inside the target bin. After each place, the object's actual world position is read and checked against the bin center (about 0.08 m tolerance) and rim height. Physical placement success and sort accuracy are tracked separately, and failed items are parked off-table so a dropped object cannot corrupt the next cycle.

12. Engineering Decisions and Notable Fixes

- Grasping: weld over friction over teleport. Friction was too fragile for small objects; teleport was unrealistic; the contact-triggered weld is robust and legitimate.
- Perception: segmentation over thresholding and over pretrained YOLO. Thresholding alone was unreliable under the lighting; pretrained YOLO does not know custom colored shapes.
- Spheres rolled away: they used condim=4 (no rolling friction). Switching to condim=6 with rolling friction (1.5 0.05 0.02) fixed it.
- Objects missing bins (decisive fix): the welded object hangs about 9 cm below the hand; at transport height 0.50 it was level with the 0.40 m walls and clipped bins it flew over. Raising transport to 0.56 raised physical placement from about 46% to about 94%.
- Cascading failures: failed grasps littered the staging area. Parking failed items and resetting the arm to the scan pose fixed the snowballing.
- Contact model: elliptic friction with high impratio caused round-object instabilities; a pyramidal cone with impratio 1 stabilized contacts.

13. Verified Results

Measured over 6 episodes of 6 randomized products each, with physical landing verified per item:

Metric	Value
Physical placement success	~94% (item lands inside the target bin)
Sort accuracy	100% (correct bin for every placed item)
Cycle time	~0.8 s per item
Failure handling	failed items auto-cleared to protect the next pick

The remaining ~6% are occasional grasp misses, which is realistic for a physical system.

14. Limitations and Future Work

Limitations:

- Simulation only, with no physical robot or sim-to-real validation.

- Detection uses the simulator's ground-truth segmentation masks; color is then classified from real RGB pixels. A learned detector is wired in but not the active path.

Future work:

- Train YOLOv8 on synthetic data with domain randomization.
- Add visual servoing on the final approach to close the remaining grasp-miss gap.
- Grasp quality estimation (antipodal analysis) to reject and re-plan poor grasps.
- A realistic depth-sensor noise model.
- Dynamic picking off a moving belt (items currently stop at a staging gate).

15. Attribution and License

The Franka Emika Panda model in assets/franka/ is from the MuJoCo Menagerie by Google DeepMind, used under its BSD-3-Clause license. Project code is released under the MIT license.